T3000 API & Protocol

Trend Logs · built Nov 13, 2026

Trend Logs

Historical data collection, analysis, and visualization.

Overview

Trend logs automatically collect and store historical data from inputs, outputs, and variables for analysis, reporting, and troubleshooting.

Trend Features

- **Real-Time Charts**: Live data visualization
- **Historical Playback**: Review past data
- **Multi-Point Comparison**: Plot multiple points
- **Custom Time Ranges**: Flexible date/time selection
- **Data Export**: CSV, Excel, JSON formats
- **Statistics**: Min, max, average, std deviation

Trend Configuration

Log Interval:

- Fast: Every 1 second
- Normal: Every 5 seconds
- Slow: Every 1 minute
- Hourly: Every hour

Data Retention:

- Real-time: 24 hours
- Historical: 1 year
- Archived: 5+ years

Using Trend Logs

1. Navigate to **Trend Logs** page

2. Select points to chart
3. Choose time range
4. View interactive charts
5. Zoom/pan to analyze
6. Export data if needed

Chart Types

- **Line Charts**: Continuous analog data
- **Step Charts**: Digital on/off states
- **Multi-Axis**: Different units on same chart
- **Comparison**: Multiple points overlaid

Analysis Features

- **Cursors**: Precise value reading
- **Statistics Panel**: Min/max/avg display
- **Annotations**: Add notes to charts
- **Alerts**: Highlight alarm conditions

Best Practices

- Log critical points continuously
- Use appropriate intervals
- Archive old data
- Review trends regularly
- Export data before archiving

Next Steps

- Trend Center Unified Design - Single-page trend workflow and phased migration plan
- Inputs - Input monitoring
- Alarms - Alarm analysis
- Monitoring - Real-time monitoring

Trend Logs

Trend Data API

Query Trend Data

TYPESCRIPT

```typescript
// Query raw trend data
const response = await fetch(
  `http://localhost:9103/api/trends?` +
  `deviceId=389001&` +
  `pointId=IN1&` +
  `start=${Date.now() - 86400000}&` +  // Last 24 hours
  `end=${Date.now()}`
);

const trends = await response.json();
// Returns: [{ timestamp, value, quality }, ...]

// Query aggregated data
const aggregated = await fetch(
  `http://localhost:9103/api/trends/aggregate?` +
  `deviceId=389001&` +
  `pointId=IN1&` +
  `start=${start}&` +
  `end=${end}&` +
  `interval=300`  // 5-minute intervals
).then(r => r.json());

// Returns: [{ timestamp, avg, min, max, count }, ...]
```

High-Performance Data Logging

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendDatabase {
  private db: Database.Database;
  private insertStmt: Database.Statement;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);

    // Optimized schema
    this.db.exec(`
      CREATE TABLE IF NOT EXISTS trends (
        timestamp INTEGER NOT NULL,
        device_id INTEGER NOT NULL,
        point_id TEXT NOT NULL,
        value REAL NOT NULL,
        quality TEXT DEFAULT 'good'
      );

      CREATE INDEX IF NOT EXISTS idx_trends_time
        ON trends(timestamp DESC);
      CREATE INDEX IF NOT EXISTS idx_trends_device_point
        ON trends(device_id, point_id, timestamp DESC);
    `);

    // WAL mode for concurrent access
    this.db.pragma('journal_mode = WAL');
    this.db.pragma('synchronous = NORMAL');

    this.insertStmt = this.db.prepare(`
      INSERT INTO trends (timestamp, device_id, point_id, value, quality)
      VALUES (?, ?, ?, ?, ?)
    `);
  }

  logBatch(entries: TrendEntry[]) {
    const transaction = this.db.transaction((entries) => {
      for (const e of entries) {
```

```
        this.insertStmt.run(
          e.timestamp,
          e.deviceId,
          e.pointId,
          e.value,
          e.quality || 'good'
        );
      }
    });

    transaction(entries);
  }
}
```

Data Aggregation

```typescript
function aggregateTrends(
  data: TrendPoint[],
  intervalMs: number
): AggregatedTrend[] {
  const buckets = new Map<number, number[]>();

  // Group by time bucket
  for (const point of data) {
    const bucket = Math.floor(point.timestamp / intervalMs) * intervalMs;

    if (!buckets.has(bucket)) {
      buckets.set(bucket, []);
    }

    buckets.get(bucket)!.push(point.value);
  }

  // Calculate statistics for each bucket
  return Array.from(buckets.entries()).map(([timestamp, values]) => ({
    timestamp,
    avg: values.reduce((a, b) => a + b, 0) / values.length,
    min: Math.min(...values),
    max: Math.max(...values),
    count: values.length,
    stdDev: calculateStdDev(values)
  }));
}
```

Next Steps

- REST API
- WebSocket API
- Performance Tuning